

Section A — SQL Basics (SELECT, Constraints, Primary Keys)

Q1. Display all columns and rows from customers table

```
SELECT * FROM customers;
```

Q2. Retrieve first_name, last_name, and city of all customers

```
SELECT first_name, last_name, city  
FROM customers;
```

Q3. List all unique categories available in products table

```
SELECT DISTINCT category  
FROM products;
```

Q4. Identify the Primary Key of each table

Table	Primary Key
customers	customer_id
products	product_id
orders	order_id
order_items	item_id

Why Primary Key must be UNIQUE and NOT NULL?

- UNIQUE ensures each record is identified separately.
- NOT NULL ensures every record has an identifier.
- Prevents duplicate and missing records.

Q5. Constraints applied to email column

```
email VARCHAR(100) UNIQUE NOT NULL
```

Constraints:

- UNIQUE → No duplicate emails allowed.
- NOT NULL → Email cannot be empty.

Example:

```
INSERT INTO customers  
VALUES (109,'Rahul','Kumar',  
'aarav.s@email.com',  
'Delhi','Delhi','2024-09-01',FALSE);
```

Result: Error because email already exists.

Q6. Insert product with negative price

```
INSERT INTO products  
VALUES (209,'Test Product','Electronics',  
'TestBrand',-50,100);
```

Error: CHECK constraint violation.

Constraint:

```
CHECK(unit_price > 0)
```

The database rejects the record because product price cannot be negative.

Section B — Filtering & Optimization (WHERE, Indexes)

Q7. Retrieve all Delivered orders

```
SELECT *  
FROM orders  
WHERE status='Delivered';
```

Q8. Electronics products with price greater than ₹2000

```
SELECT *  
FROM products  
WHERE category='Electronics'  
AND unit_price > 2000;
```

Q9. Customers from Maharashtra who joined in 2024

```
SELECT *  
FROM customers  
WHERE state='Maharashtra'  
AND join_date BETWEEN '2024-01-01'  
AND '2024-12-31';
```

Q10. Orders between 2024-08-10 and 2024-08-25 excluding cancelled

```
SELECT *  
FROM orders  
WHERE order_date BETWEEN '2024-08-10'  
AND '2024-08-25'  
AND status <> 'Cancelled';
```

Q11. Purpose of idx_orders_date

```
CREATE INDEX idx_orders_date  
ON orders(order_date);
```

Purpose:

- Speeds up searching orders by date.
- Database finds matching rows quickly instead of scanning the entire table.

Example:

```
SELECT *  
FROM orders  
WHERE order_date BETWEEN '2024-08-01'  
AND '2024-08-31';
```

Q12. Will index be used?

Query:

```
SELECT *  
FROM customers  
WHERE YEAR(join_date)=2024;
```

No.

Because function YEAR() is applied on the indexed column, making it non-SARGable.

Index-friendly version:

```
SELECT *  
FROM customers  
WHERE join_date >= '2024-01-01'  
AND join_date < '2025-01-01';
```

Section C — Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

Q13. Count total orders

```
SELECT COUNT(*) AS total_orders  
FROM orders;
```

Q14. Total revenue from Delivered orders

```
SELECT SUM(total_amount) AS total_revenue  
FROM orders  
WHERE status='Delivered';
```

Q15. Average product price by category

```
SELECT category,  
AVG(unit_price) AS avg_price  
FROM products  
GROUP BY category;
```

Q16. Order count and revenue by status

```
SELECT status,  
COUNT(*) AS order_count,  
SUM(total_amount) AS total_revenue  
FROM orders  
GROUP BY status  
ORDER BY total_revenue DESC;
```

Q17. Most expensive and cheapest product in each category

```
SELECT category,  
MAX(unit_price) AS highest_price,  
MIN(unit_price) AS lowest_price  
FROM products  
GROUP BY category;
```

Q18. Categories with average price greater than ₹2000

```
SELECT category,  
AVG(unit_price) AS avg_price  
FROM products  
GROUP BY category  
HAVING AVG(unit_price) > 2000;
```

Section D — Joins & Relationships

Q19. INNER JOIN orders with customers

```
SELECT o.order_id,  
       o.order_date,  
       c.first_name,  
       c.last_name,  
       o.total_amount  
FROM orders o  
INNER JOIN customers c  
ON o.customer_id = c.customer_id;
```

Q20. LEFT JOIN all customers with orders

```
SELECT c.customer_id,  
       c.first_name,  
       c.last_name,  
       o.order_id,  
       o.order_date,  
       o.total_amount  
FROM customers c  
LEFT JOIN orders o  
ON c.customer_id = o.customer_id;
```

Q21. Join three tables

```
SELECT o.order_id,  
       p.product_name,  
       oi.quantity,  
       oi.unit_price,  
       oi.discount_pct  
FROM orders o  
JOIN order_items oi  
ON o.order_id = oi.order_id  
JOIN products p  
ON oi.product_id = p.product_id;
```

Q22. Difference between LEFT JOIN and RIGHT JOIN

LEFT JOIN

- Returns all rows from left table.
- Matching rows from right table.

```
SELECT *  
FROM customers c
```

```
LEFT JOIN orders o
ON c.customer_id=o.customer_id;
```

RIGHT JOIN

- Returns all rows from right table.
- Matching rows from left table.

```
SELECT *
FROM customers c
RIGHT JOIN orders o
ON c.customer_id=o.customer_id;
```

FULL OUTER JOIN

- Returns all records from both tables.
- Matching and non-matching rows.

Used when you want complete data from both tables.

Q23. Foreign Key relationships

Parent Table	Child Table
customers.customer_id	orders.customer_id
orders.order_id	order_items.order_id
products.product_id	order_items.product_id

Example:

```
INSERT INTO orders
VALUES (1011,999,'2024-09-01',
'Pending',1000);
```

Result: Error because customer_id 999 does not exist in customers table.

Section E — Advanced Concepts (CASE, ACID, Transactions)

Q24. Classify products by price tier

```
SELECT product_name,  
       unit_price,  
       CASE  
         WHEN unit_price < 1000  
         THEN 'Budget'  
  
         WHEN unit_price BETWEEN 1000 AND 3000  
         THEN 'Mid-Range'  
  
         ELSE 'Premium'  
       END AS price_tier  
FROM products;
```

Q25. Count Delivered vs Not Delivered

```
SELECT  
  SUM(  
    CASE  
      WHEN status='Delivered'  
      THEN 1  
      ELSE 0  
    END  
  ) AS delivered_orders,  
  
  SUM(  
    CASE  
      WHEN status<>'Delivered'  
      THEN 1  
      ELSE 0  
    END  
  ) AS not_delivered_orders  
FROM orders;
```

Q26. ACID Properties

A – Atomicity

Either all operations succeed or none.

Example:

- Money deducted from Account A and added to Account B.

- If one step fails, both are cancelled.

C – Consistency

Database remains valid before and after transaction.

Example:

- Total money in system remains correct.

I – Isolation

Transactions do not interfere with each other.

Example:

- Two users transferring money simultaneously should not affect each other's transaction.

D – Durability

Once committed, data remains saved permanently.

Example:

- After transfer succeeds, data remains even if power fails.

Q27. Transaction Block

```
START TRANSACTION;
```

```
INSERT INTO orders
(order_id, customer_id,
order_date, status, total_amount)
VALUES
(1011,102,CURDATE(),
'Pending',1598.00);
```

```
INSERT INTO order_items
(item_id,order_id,product_id,
quantity,unit_price,discount_pct)
VALUES
(5016,1011,206,1,1299.00,0);
```

```
INSERT INTO order_items
(item_id,order_id,product_id,
quantity,unit_price,discount_pct)
VALUES
(5017,1011,208,1,599.00,0);
```

```
UPDATE products  
SET stock_qty = stock_qty - 1  
WHERE product_id = 206;
```

```
UPDATE products  
SET stock_qty = stock_qty - 1  
WHERE product_id = 208;
```

```
COMMIT;
```

If any statement fails:

```
ROLLBACK;
```

This ensures the entire transaction is completed successfully or completely undone.